

UNIVERZITET U BEOGRADU
MATEMATIČKI FAKULTET



Vukan Antić

UPOREDJIVANJE PROTOKOLA ZA
KOMUNIKACIJU GRAPHQL, GRPC I REST,
RAZVOJOM MOBILNE APLIKACIJE
UPOTREBOM TEHNOLOGIJA SPRING BOOT
I REACT NATIVE

master rad

Beograd, 2026.

Mentor:

dr Vladimir FILIPOVIĆ, redovan profesor
Univerzitet u Beogradu, Matematički fakultet

Članovi komisije:

dr Aleksandar KARTELJ, vandredni profesor
Univerzitet u Beogradu, Matematički fakultet

dr Staša VUJIČIĆ STANKOVIĆ, docent
Univerzitet u Beogradu, Matematički fakultet

Datum odbrane: _____

*Ovaj rad posvećujem svom ocu, majci, bratu,
prijateljima i kolegama na podršci u toku izrade ovog
rada. dr. Vladimiru Filipoviću se zahvaljujem na
strpljenju kao i savetima u toku izgrada ovog rada.
Posebnu zahvalnost dugujem partneru Dušici, na svoj
ljubavi pruženoj u toku studija.*

Naslov master rada: Uporedjivanje protokola za komunikaciju GraphQL, GRPC i Rest, razvojem mobilne aplikacije upotrebom tehnologija Spring Boot i React Native

Rezime: TODO popuni kasnije

Ključne reči: TODO popuni kasnije

Sadržaj

1	Uvod	1
2	Prikupljivanje podataka	3
2.1	Obrada podataka	4
2.1.1	Filtriranje	5
2.1.2	Provera ispravnosti slike	6
2.1.3	Relacioni model	6
2.1.4	Upisivanje u bazu	8
3	Server	10
4	Klijent	11
5	Protokoli komunikacije	12
	Bibliografija	13

Glava 1

Uvod

Sa rastućom upotrebom mobilnih aplikacija, komunikacija između klijenta i servera se u većini slučajeva svede na Rest [1] protokol komunikacije iz navike, iako postoje i drugi protokoli poput GRPC [2] i GraphQL [3], koji su performantniji, fleksibilniji, kao i skalabilniji. Zato je u toku izrade naredne aplikacije bila upotreba sva 3 protokola, da bi se moglo videti kada je koji protokol najbolje koristiti.

Sam projekat predstavlja aplikaciju otvorenog koda¹, koja svakodnevno daje korisniku umetničku sliku, sa detaljnim opisom, i daje mu mogućnost da sačuva omiljenu, radi dalje preporučivanje novog sadržaja. Pored toga, postoje i funkcionalnosti čuvanja omiljenih slika u memoriji sopstvenog telefona, kao i da deli slike sa drugim korisnicima. Takođe, pri izlasku nove slike, korisnik dobija obaveštenje o novom sadržaju koji je dodat za njega.

Izrada projekta čine sledeći delovi:

- Prikupljivanje podataka - U glavi 2 predstavljen je detaljan opis odakle je aplikacija dobila podatke za prikaz u okrivu same aplikacije.
- Izrada servera zasnovanog na mikroservisnoj arhitekturi [4] - U glavi 3 predstavljena implementacija servera, koja je zasnovana na mikroservison arhitekturi, dok je svaki od mikroservisa pratio (eng. *Onion*) arhitekturu [5, 6], koja je pomogla da se lako dodaje podrška za sve protokole komunikacije.
- Izrada klijenta zasnovanog na MVC arhitekturi [7] - U glavi 4 prikazano je kako je implementiran klijentski deo koda, koja prati MVC arhitekturu, kao i dodatne funkcionalnosti koje klijent podržava, kao što je videt (eng. *widget*).

¹<https://github.com/VukanAntic/ArtOfTheDay>

- Dodavanje GPRC i GraphQL - Glava 5 predstavlja prikaz svih gore navedenih protokola komunikacije, kao i poređenja.

Glava 2

Prikupljivanje podataka

Aplikacija razvijena u okviru ovog rada, *INSPIRA daily*, prikazuje korisniku novu umetničku sliku svakog dana uzimajući u obzir preference korisnika. Potrebe projekta su bile da ima veliku količinu umetničkih slika, detaljan opis za svaku, žanr kojoj slika prikada, kao i informacija ko je bio slikar iste. Iz tih razloga, za izvor podataka bio je izabran Institut za umetnost u Čikagu (eng. *The Art Institute of Chicago*) [8]. Institut nudi javni programski interfejs, ali i potpuni prepis zbirke (eng. *data dump*) - arhiva svih slika, sačuvanih u JSON obliku. Odabran je pristup preko arhive, iz razloga što bi dohvaćanje podataka preko interfejsa zahtevalo veliki vremenski period, kao i stavljanje velikog pritiska na server instituta.

Slike se ne čuvaju lokalno, iz razloga jer ih institut izlaže preko protokola IIIF (eng. *International Image Interoperability Framework*) koji omogućava da se željena slika, kao i njega širina zatraži preko adrese (*url*). Adresa slike se može sastaviti iz identifikatora slike (eng. *image id*), što pokazuje Primer koda 2.1.

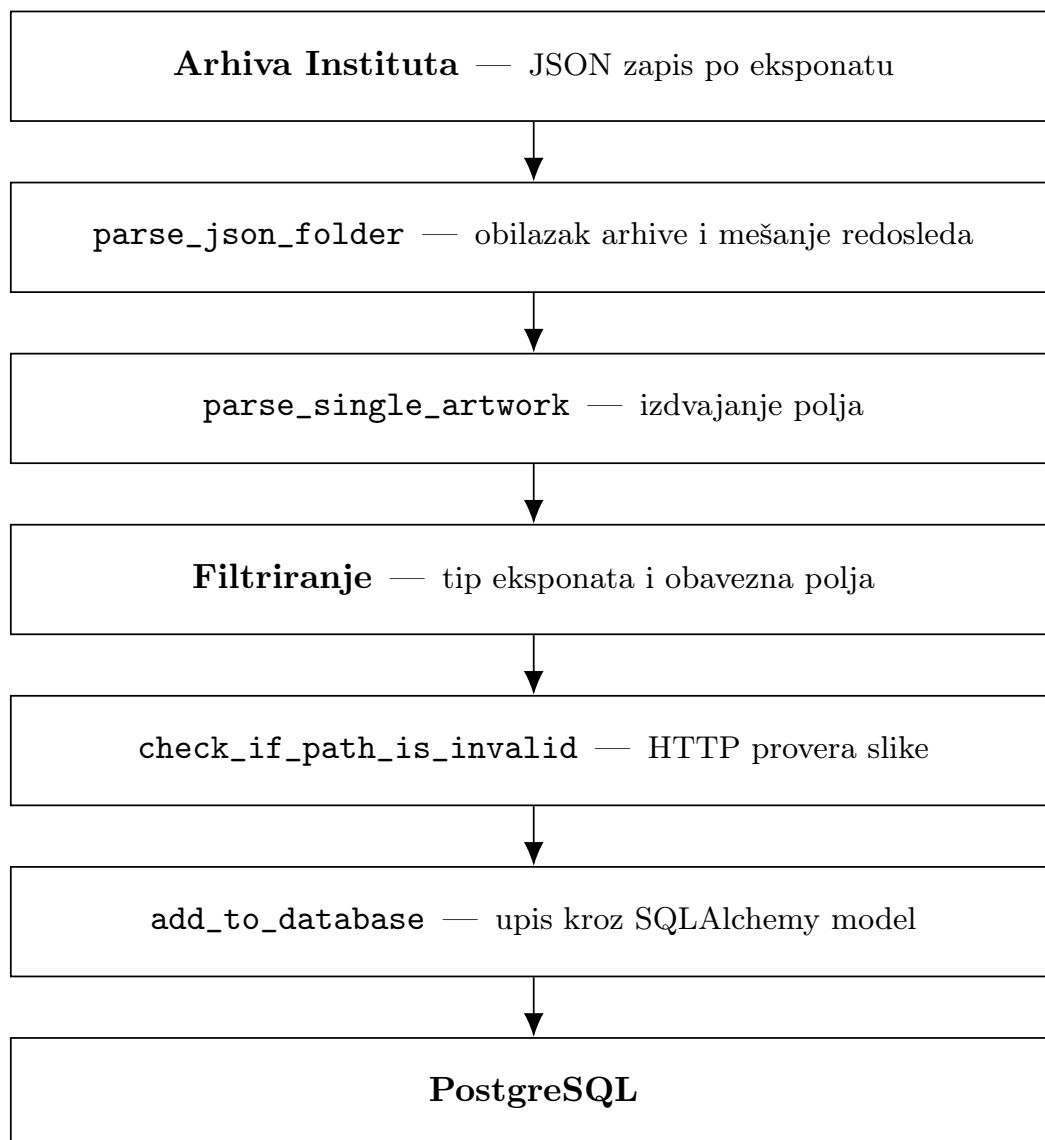
```
1 path = f'https://www.artic.edu/iiif/2/{data["image_id"]}'  
2      /full/843,/0/default.jpg'
```

Primer koda 2.1: Sastavljanje adrese slike iz identifikatora

Na ovaj način, za dohvaćanje i prikazivanje slika bilo je samo potrebno izvući identifikator svake željene slike, i dobija se adresa iste. Vrednost 843 podrazumeva najčešću korišćenu veličinu slike, koja je verovatno keširana na strani institutovog servera. Bilo je moguće zameniti tu vrednost sa manjom vrednošću, da sve slike imaju istu širinu i visinu.

2.1 Obrada podataka

Automatska obrada podataka, bila je realizovana uz pomoć skripte napisane u programskom jeziku *Python* po imenu *ImageDatabaseGenerator.py*. Slika 2.1 prikazuje kako je to urađeno.



Slika 2.1: Tok obrade podataka

2.1.1 Filtriranje

Arhiva instituta ne sadrži samo umetničke slike, već i skulpture, fotografije, tekstil kao i veliku količinu drugih tipova objekata. Iz tog razloga, bilo je potrebno filtrirati podatke, tako da se pronađu samo umetničke slike, što se može videti Primer koda 2.2.

```
1 def parse_single_artwork(data):
2     id = data["id"]
3     title = data["title"]
4     description = data["description"]
5     date_display = data["date_display"]
6     style_id = data["style_id"]
7     style_title = data["style_title"]
8     artist_id = data["artist_id"]
9     artist_title = data["artist_title"]
10    artwork_type_id = data["artwork_type_id"]
11    # id = 1 represents artworks
12    artworked_wanted_indicies = [1]
13    path = f'https://www.artic.edu/iiif/2/{data["image_id"]}/full
14           /843,/0/default.jpg'
15    if title is None or description is None or date_display is None
16       \
17    or style_title is None or artwork_type_id not in
18       artworked_wanted_indicies or \
19    check_if_path_is_invalid(path) :
20        return None
21    return {
22        "id" : id,
23        "title" : title,
24        "description" : description,
25        "date_display" : date_display,
26        "path" : path,
27        "style_title" : style_title,
28        "style_id" : style_id,
29        "artist_title" : artist_title,
30        "artist_id" : artist_id
31    }
```

Primer koda 2.2: Filtriranje umetničkih slika

Pored provere da li je u pitanju umetnička slika, bilo je potrebno dodati i druge

provere da li postoji naziv slike, opis, i datum njene kreacije.

2.1.2 Provera ispravnosti slike

Naredni korak posle filtriranja podrazumeva provera da li slika koju imaju u arhivi instituta ispravna, tj. da li pristup adresi te slike vraća grešku, što pokazuje primer koda 2.3.

```
1 def check_if_path_is_invalid(path) :
2     response = requests.get(path)
3     print(path)
4     print(response.status_code)
5
6     if response.status_code == 429:
7         print("Too many requests in a short period of time, need to
8             cool off...")
9         time.sleep(60)
10        response = requests.get(path)
11    if response.status_code == 200:
12        return False
13
14    print("Found an error code!")
15    return True
```

Primer koda 2.3: Provera dostupnosti

Bitno je napomenuti, programski interfejs koji je dostupan od instituta vraća grešku **429**, ako je previše zahteva stiglo od iste *IP adrese* u kratkom vremenskom periodu, zato u toku izvršavanje skripte, zaustavi se program na 60 sekundi, da bi moglo dalje da se pristupi serveru instituta.

2.1.3 Relacioni model

Izdvojeni podaci upisuju se u bazu (eng. *PostgreSQL*) preko biblioteke (eng. *SQLAlchemy*). Potrebni podaci se čuvaju u tri različite tabele - Umetnička slika (eng. *Artwork*), Žanr (eng. *Genre*) kao i Umetnik (eng. *Artist*). Kod 2.4 koji generiše takav model možemo videti:

```
1 artwork_genre_association = Table(
```

```

2     'artwork_genre', Base.metadata,
3     Column('artwork_id', Integer, ForeignKey('artworks.id')),
4     Column('genre_id', String(20), ForeignKey('genres.id'))
5 )
6
7 class Artwork(Base):
8     __tablename__ = 'artworks'
9     id = Column(Integer, primary_key=True, autoincrement=True)
10    title = Column(String(255), nullable=False)
11    date_display = Column(String(255), nullable=False)
12    description = Column(Text, nullable=False)
13    path_to_artwork = Column(String(255), nullable=False)
14    artist_id = Column(Integer, ForeignKey('artists.id'), nullable=
        False)
15
16    # Relationships
17    genres = relationship('Genre', secondary=
        artwork_genre_association, back_populates='artworks')
18    artist = relationship('Artist', back_populates='artworks')
19
20 class Artist(Base):
21     __tablename__ = 'artists'
22
23     id = Column(Integer, primary_key=True, autoincrement=True)
24     # Unknown artist!
25     name = Column(String(255), nullable=True)
26
27     artworks = relationship('Artwork', back_populates='artist')
28
29 class Genre(Base):
30     __tablename__ = 'genres'
31
32     id = Column(String(20), primary_key=True)
33     name = Column(String(255), nullable=False)
34
35     # Define the relationship to Artwork
36     artworks = relationship('Artwork', secondary=
        artwork_genre_association, back_populates='genres')

```

Primer koda 2.4: Relacioni model

Tri bitne stavke o samom modelu koje je značajno izdvojiti.

- **Identifikatori se preuzimaju od izvora** - Delo i autor zadržavaju celobrojni identifikator koji im je dodelio institut, umesto da dobiju novi. Time je omogućeno da se u budućnosti zbirka dopuni novim slikama i umetnicima, bez potrebe za brigom o duplikatima. Ista odluka primenjena je i na žanrove, identifikator je zadržan od instituta, koji je niska.
- **Veza slika-žanr je više-prema-više** - Zbog prirode skupa podataka, jedna slika može pripadati više različitih žanrova u isto vreme, i isto tako, jednom žanru može odgovarati veći broj slika.
- **Veza slika-umetnik je jedan-prema-više** - Isto kao i za prethodni odnos, zbog prirode podataka, jednu sliku je mogao da naslika jedan umetnik, dok je jedan umetnik mogao da naslika više različitih slika.

2.1.4 Upisivanje u bazu

Nakon što se uspešno kreira model, prelazi se na naredni korak, upisivanje podataka koji su uzeti iz arhive u novo kreiranu bazu. U dole navedenom kodu 2.5 možemo videti da, nakon što su entiteti kreirani i dodati u liste, u toku jedne izmene (eng. *commit*), podaci su dodati u bazu podataka.

```
1         if artist_id not in artist_ids:
2             artists_data.append(artist_data)
3         if artwork["style_id"] not in genre_ids:
4             genres_data.append(genre_data)
5
6         new_artist_data = list(filter(lambda elem : elem.id ==
7                                     artist_id, artists_data))
8         new_genre_data = list(filter(lambda elem : elem.id ==
9                                     artwork["style_id"], genres_data))
10        print(new_artist_data)
11        artwork_data.artist = new_artist_data[0]
12        artwork_data.genres.append(new_genre_data[0])
13
14        artwork_ids.add(artwork["id"])
15        artist_ids.add(artist_id)
16        genre_ids.add(artwork["style_id"])
17
18        session.add_all(artists_data)
```

```
18     session.add_all(genres_data)
19     session.add_all(artworks_data)
20     session.commit()
```

Primer koda 2.5: Isecak koda koji upisuje podatke u bazu podataka

Glava 3

Server

Glava 4

Klijent

Glava 5

Protokoli komunikacije

Bibliografija

- [1] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. online at: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [2] gRPC Authors. gRPC Documentation: Core Concepts, Architecture and Lifecycle. <https://grpc.io/docs/what-is-grpc/core-concepts/>, 2024. Pristupljeno: 1. avgusta 2026.
- [3] GraphQL Foundation. GraphQL Specification. <https://spec.graphql.org/>, 2021. Pristupljeno: 1. avgusta 2026.
- [4] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2 edition, 2021.
- [5] Jeffrey Palermo. The Onion Architecture: Part 1. <https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/>, 2008. Pristupljeno: 1. avgusta 2026.
- [6] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [7] Martin Fowler. Model View Controller. <https://martinfowler.com/eaCatalog/modelViewController.html>, 2002. Pristupljeno: 1. avgusta 2026.
- [8] The Art Institute of Chicago. Art Institute of Chicago API. <https://api.artic.edu/docs/>, 2025. Pristupljeno: 15. marta 2025.